Observability and Traffic Engineering

Istio Service Mesh on Bookinfo

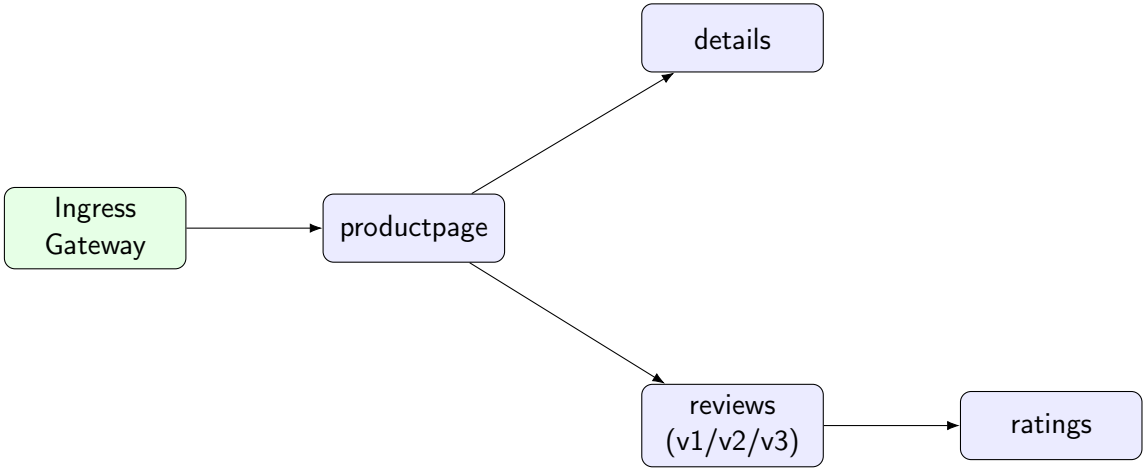
Diego, Esteban, Emmanuel

June 21, 2026

Agenda

1. Observability Stack
2. Traffic Engineering
3. Live Demonstration

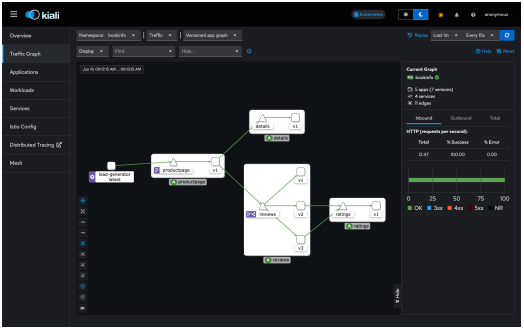
Mesh Architecture Under Observation



Every hop is an Envoy sidecar call: Prometheus scrapes metrics, Jaeger collects spans, and Kiali renders the live graph from both.

Kiali: Service Topology

- Renders the live traffic graph from Prometheus data, refreshed every 15s.
- Shows per-edge request rate and success %.
- Visualizes weighted routing (canary split) and header-based edges directly on the graph.
- **Istio Config** view lists every CRD (VirtualService, DestinationRule, Gateway, PeerAuthentication, Telemetry) with validation status.

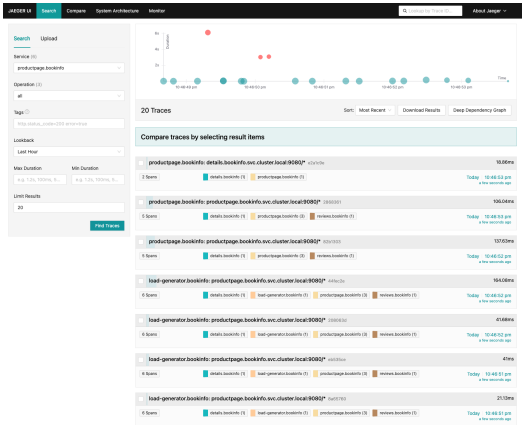


Grafana: Metrics Dashboards

- `06-install-metrics.sh` installs Prometheus and a custom **Bookinfo** dashboard.
- Three panels: request rate per service, 5xx error rate, and per-version `reviews` request rate.
- Standard Istio addon dashboards (Mesh, Service, Control Plane) also available.
- Used throughout the project to evidence canary splits and fault-injection scenarios.

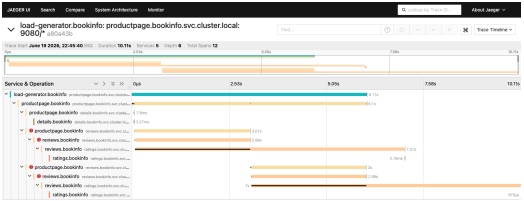
Jaeger: Distributed Tracing

- `07-install-tracing.sh` installs single-binary Jaeger plus a `Telemetry` resource setting **100% trace sampling** in `bookinfo` (vs. Istio's 1% default).
- Services propagate W3C/B3 headers end-to-end, so one trace spans every hop.
- Search view: every request entering at `productpage`, fanning out to `details` and `reviews`.



Jaeger: Trace Waterfall

- Full call tree: `load-generator` → `productpage` → {`details`, `reviews` → `ratings`}.
- 5 services, 12 spans, per-span timing.
- Pinpoints exactly which downstream call is slow or failing — e.g. a delayed `reviews` → `ratings` span surfaces immediately as an outlier.



Traffic Engineering: Overview

- Goal: control how requests to `reviews` are split across `v1`/`v2`/`v3` without touching application code.
- Two complementary mechanisms, both expressed as Istio `VirtualService` resources:
  1. **Canary release** — weighted traffic split.
  2. **Header-based routing** — deterministic routing for a specific user.
- Combined into a single `VirtualService` with priority-ordered HTTP match rules.

Canary Release: 90/5/5 Split

`networking/virtual-service-canary.yaml`

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews-canary
spec:
  hosts: [reviews]
  http:
  - route:
    - destination: {host: reviews, subset: v1}
      weight: 90
    - destination: {host: reviews, subset: v2}
      weight: 5
    - destination: {host: reviews, subset: v3}
      weight: 5
```

`scripts/10-verify-canary.sh` samples 100 requests and asserts the v2+v3 ratio falls within $10\% \pm 5\%$.

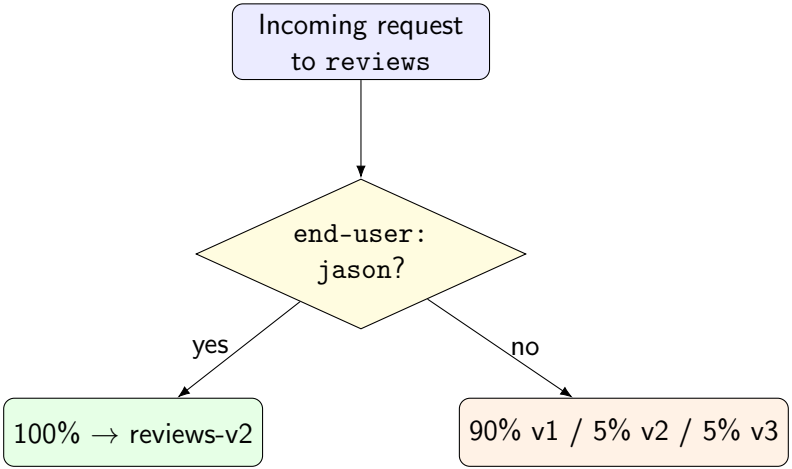
Header-Based Routing

`networking/combined-virtual-service.yaml` — priority order:

1. Header `end-user: jason` → 100% to `reviews-v2`.
2. Everything else → falls through to the 90/5/5 canary split.

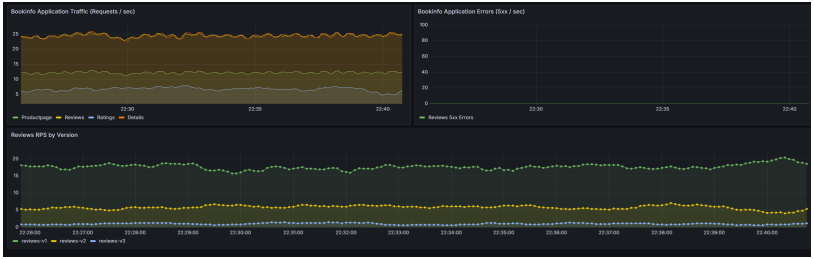
```
http:
  - name: header-based-routing
    match:
    - headers: {end-user: {exact: "jason"}}
    route:
    - destination: {host: reviews, subset: v2}
      weight: 100
  - name: canary-default-routing
    route:
    - destination: {host: reviews, subset: v1}
      weight: 90
    - destination: {host: reviews, subset: v2}
      weight: 5
    - destination: {host: reviews, subset: v3}
      weight: 5
```

Routing Decision Path



`scripts/11-test-header-routing.sh` confirms both paths: an anonymous request gets any valid version, a `jason` request always gets v2.

Observed Effect: Kiali Traffic Split



Kiali renders both the weighted edges to v1/v2/v3 and the header-matched edge to v2 under load.

Live Demonstration

1. Cluster, Istio, Bookinfo, metrics, and tracing already running (`scripts/run-all.sh`, lines 1–37).
2. Apply the **canary** `VirtualService` and run `10-verify-canary.sh`.
3. Apply the **combined header-based** `VirtualService` and run `11-test-header-routing.sh`.
4. Observe the effect live in **Kiali**, **Grafana**, and **Jaeger**.

No further slides — switching to the terminal and browser.

Thank You

Questions?